7.2 Increment and Decrement Operators

[This section corresponds to K&R Sec. 2.8]

The assignment operators of the previous section let us replace *v = v op e* with *v op= e*, so that we didn't have to mention *v* twice. In the most common cases, namely when we're adding or subtracting the constant 1 (that is, when *op* is + or - and *e* is 1), C provides another set of shortcuts: the *autoincrement* and *autodecrement* operators. In their simplest forms, they look like this:

```
++i             add 1 to i
--j             subtract 1 from j
```

These correspond to the slightly longer `i += 1` and `j -= 1`, respectively, and also to the fully ``longhand'' forms `i = i + 1` and `j = j - 1`.

The `++` and `--` operators apply to one operand (they're *unary* operators). The expression `++i` adds 1 to `i`, and stores the incremented result back in `i`. This means that these operators don't just compute new values; they also modify the value of some variable. (They share this property--modifying some variable--with the assignment operators; we can say that these operators all have *side effects*. That is, they have some effect, on the side, other than just computing a new value.)

The incremented (or decremented) result is also made available to the rest of the expression, so an expression like

```
k = 2 * ++i
```

means ``add one to `i`, store the result back in `i`, multiply it by 2, and store *that* result in `k`." (This is a pretty meaningless expression; our actual uses of `++` later will make more sense.)

Both the `++` and `--` operators have an unusual property: they can be used in two ways, depending on whether they are written to the left or the right of the variable they're operating on. In either case, they increment or decrement the variable they're operating on; the difference concerns whether it's the old or the new value that's ``returned'' to the surrounding expression. The *prefix* form `++i` increments `i` and returns the incremented value. The *postfix* form `i++` increments `i`, but returns the *prior*, non-incremented value. Rewriting our previous example slightly, the expression

```
k = 2 * i++
```

means ``take `i`'s old value and multiply it by 2, increment `i`, store the result of the multiplication in `k`."

The distinction between the prefix and postfix forms of `++` and `--` will probably seem strained at first, but it will make more sense once we begin using these operators in more realistic situations.

For example, our `getline` function of the previous chapter used the statements

```
line[nch] = c;
nch = nch + 1;
```

as the body of its inner loop. Using the `++` operator, we could simplify this to

```
line[nch++] = c;
```

We wanted to increment `nch` *after* deciding which element of the `line` array to store into, so the postfix form `nch++` is appropriate.

Notice that it only makes sense to apply the `++` and `--` operators to variables (or to other ``containers," such as `a[i]`). It would be meaningless to say something like

```
1++
```

or

```
(2+3)++
```

The `++` operator doesn't just mean ``add one"; it means ``add one *to a variable*" or ``make a variable's value one more than it was before." But `(1+2)` is not a variable, it's an expression; so there's no place for `++` to store the incremented result.

Another unfortunate example is

```
i = i++;
```

which some confused programmers sometimes write, presumably because they want to be extra sure that `i` is incremented by 1. But `i++` all by itself is sufficient to increment `i` by 1; the extra (explicit) assignment to `i` is unnecessary and in fact counterproductive, meaningless, and incorrect. If you want to increment `i` (that is, add one to it, and store the result back in `i`), either use

```
i = i + 1;
```

or

```
i += 1;
```

or

```
++i;
```

or

```
i++;
```

Don't try to use some bizarre combination.

Did it matter whether we used `++i` or `i++` in this last example? Remember, the difference between the two forms is what value (either the old or the new) is passed on to the surrounding expression. If there is no surrounding expression, if the `++i` or `i++` appears all by itself, to increment `i` and do nothing else, you can use either form; it makes no difference. (Two ways that an expression can appear ``all by itself," with ``no surrounding expression," are when it is an expression statement terminated by a semicolon, as above, or when it is one of the controlling expressions of a `for` loop.) For example, both the loops

```
for(i = 0; i < 10; ++i)
        printf("%d\n", i);
```

and

```
for(i = 0; i < 10; i++)
        printf("%d\n", i);
```

will behave exactly the same way and produce exactly the same results. (In real code, postfix increment is probably more common, though prefix definitely has its uses, too.)

In the preceding section, we simplified the expression

```
a[d1 + d2] = a[d1 + d2] + 1;
```

from a previous chapter down to

```
a[d1 + d2] += 1;
```

Using ++, we could simplify it still further to

```
a[d1 + d2]++;
```

or

```
++a[d1 + d2];
```

(Again, in this case, both are equivalent.)

We'll see more examples of these operators in the next section and in the next chapter.

Read sequentially: prev next up top